

Process Synchronization

When two or more process cooperates with each other, their order of execution must be preserved otherwise there can be conflicts in their execution and inappropriate outputs can be produced.

A cooperative process is the one which can affect the execution of other process or can be affected by the execution of other process. Such processes need to be synchronized so that their order of execution can be guaranteed.

Producer-Consumer problem

The Producer-Consumer problem is a classical multi-process synchronization problem, that is we are trying to achieve synchronization between more than one process.

There is one Producer in the producer-consumer problem, Producer is producing some items, whereas there is one Consumer that is consuming the items produced by the Producer. The same memory buffer is shared by both producers and consumers which is of fixed-size.

The task of the Producer is to produce the item, put it into the memory buffer, and again start producing items. Whereas the task of the Consumer is to consume the item from the memory buffer.

Let's understand what is the problem?

Below are a few points that considered as the problems occur in Producer-Consumer:

- o The producer should produce data only when the buffer is not full. In case it is found that the buffer is full, the producer is not allowed to store any data into the memory buffer.

Data can only be consumed by the consumer if and only if the memory buffer is not empty. In case it is found that the buffer is empty, the consumer is not allowed to use any data from the memory buffer. Accessing memory buffer should not be allowed to producer and consumer at the same time.

Let's see the code for the above problem:

Producer Code

```
int count = 0;
void producer( void )
{
    int itemP;
    while(1)
    {
        Produce_item(item P)
        while( count == n);
        buffer[ in ] = item P;
        in = (in + 1)mod n
        count = count + 1;
    }
}
```



Load Rp, m[count]
Increment Rp
Store m[count], Rp

Consumer code

```
int count;
void consumer(void)
{
    int itemC;
    while( 1 )
    {
        while( count == 0 );
        itemC = buffer[ out ];
        out = ( out + 1 ) mod n;
        count = count - 1;
    }
}
```



```
Load Rc, m[count]
Decrement Rc
Store m[count], Rc
```

Let's understand above Producer and Consumer code:

Before Starting an explanation of code, first, understand the few terms used in the above code:

1. "**in**" used in a producer code represent the next **empty buffer**
2. "**out**" used in consumer code represent first **filled buffer**
3. count keeps the count number of elements in the buffer
4. count is further divided into 3 lines code represented in the block in both the producer and consumer code.

If we talk about Producer code first:

- Rp is a register which keeps the value of m[count]
- Rp is incremented (As element has been added to buffer)
- an Incremented value of Rp is stored back to m[count]

Similarly, if we talk about Consumer code next:

- Rc is a register which keeps the value of m[count]
- Rc is decremented (As element has been removed out of buffer)
- the decremented value of Rc is stored back to m[count].

BUFFER

```
(pointer to the next empty space)→in    = 5
(pointer to the first filled space)→out   = 0
(number of elements in Buffer)→count     = 5
(Size of Buffer)→n                       = 8
```

0	A
1	B
2	C
3	D
4	E
5	
6	
7	

As we can see from Fig: Buffer has total 8 spaces out of which the first 5 are filled, $in = 5$ (pointing next empty position) and $out = 0$ (pointing first filled position).

Let's start with **the producer** who wanted to produce an element " F ", according to code it will enter into the `producer()` function, `while(1)` will always be true, `itemP = F` will be tried to insert into the buffer, before that

`while(count == n);` will evaluate to be False.

Note: Semicolon after while loop will not let the code to go ahead if it turns out to be True(i.e. infinite loop/ Buffer is full)

`Buffer[in] = itemP` $\rightarrow$ `Buffer[5] = F`. (F is inserted now)

$in = (in + 1) \bmod n \rightarrow (5 + 1) \bmod 8 \rightarrow 6$, therefore $in = 6$; (next empty buffer)

After insertion of F, Buffer looks like this

Where **out = 0**, but **in = 6**

0	A
1	B
2	C
3	D
4	E
5	F
6	
7	

Since `count = count + 1;` is divided into three parts:

Load `Rp, m[count]` $\rightarrow$ will copy count value which is 5 to register `Rp`.

Increment `Rp` $\rightarrow$ will increment `Rp` to 6.

Suppose just after Increment and before the execution of third line (store m[count], Rp) **Context Switch** occurs and code jumps to **consumer code**. . .

Consumer Code:

Now starting **consumer** who wanted to consume the first element " A ", according to code it will enter into the consumer() function, while(1) will always be true, while(count == 0); will evaluate to be False(since the count is still 5, which is not equal to 0.

Note: Semicolon after while loop will not let the code to go ahead if it turns out to be True(i.e. infinite loop/ no element in buffer)

itemC = Buffer[out] → itemC = A (since out is 0)

out = (out + 1) mod n → (0 + 1) mod 8 → 1, therefore out = 1(first filled position)

A is removed now

After removal of A, Buffer look like this

Where **out = 1**, and **in = 6**

0	
1	B
2	C
3	D
4	E
5	F
6	
7	

Since count = count - 1; is divided into three parts:

Load Rc, m[count] → will copy count value which is 5 to register Rp.

Decrement Rc → will decrement Rc to 4.

store m[count], Rc → count = 4.

Now the current value of count is 4

Suppose after this **Context Switch** occurs back to the leftover part of producer code. . .

Since context switch at producer code was occurred after Increment and before the execution of the third line (store m[count], Rp)

So we resume from here since Rp holds 6 as incremented value

Hence store m[count], Rp $\rightarrow$ count = 6

Now the current value of count is 6, which is wrong as Buffer has only 5 elements, this condition is known as Race Condition and Problem is Producer-Consumer Problem

Race condition

Two types of process categories

- **Independent process:-** Two processes are said to be independent if the execution of one process does not affect the execution of another process.
- **Cooperating process:** Two processes are said to be cooperative if the execution of one process affects the execution of another process.
- In this case, the execution of one process affects execution of another process because these processes share something. Like they share variable, memory, code, resources (CPU, Printer, Scanner etc).

These cooperative processes needs to be synchronized to generate correct **result after execution**

It occurs when two or more processes are executed at same time , not scheduled in proper sequence & not executed in critical section correctly.

Which causes data inconsistency & data loss.

Process synchronization :- It helps to maintain shared data consistency & cooperating process, Execution when process execute it should be ensure that concurrent access to shared data, does not create inconsistency

P1	P2
{	{
Read(A);	Read(A);
A=A*2;	A=A+5;
Write(A);	Write(A);
}	}

The Critical Section Problem

Critical Section is the part of a program which tries to access shared resources. That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device.

The critical section cannot be executed by more than one process at the same time; operating system faces the difficulties in allowing and disallowing the processes from entering the critical section.

The critical section problem is used to design a set of protocols which can ensure that the Race condition among the processes will never arise.

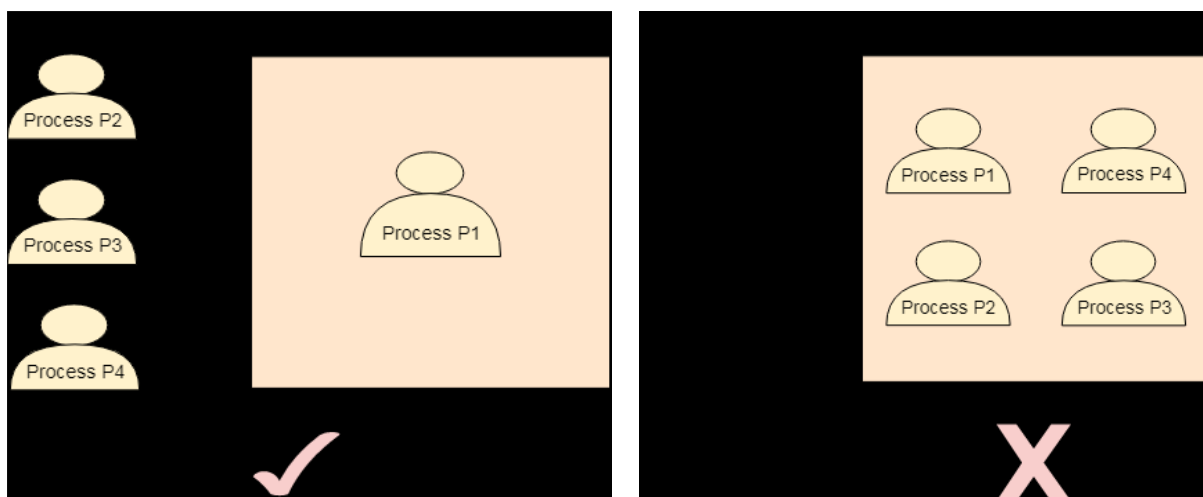
In order to synchronize the cooperative processes, our main task is to solve the critical section problem. We need to provide a solution in such a way that the following conditions can be satisfied

Requirements of Synchronization mechanisms

Primary

1. Mutual Exclusion

Our solution must provide mutual exclusion. By Mutual Exclusion, we mean that if one process is executing inside critical section then the other process must not enter in the critical section.



2. **Progress** Progress means that if one process doesn't need to execute into critical section then it should not stop other processes to get into the critical section.

Secondary

1. **Bounded Waiting** We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.

2. **Architectural Neutrality** Our mechanism must be architectural natural. It means that if our solution is working fine

on one architecture then it should also run on the other ones as well.

Semaphore:

It is a synchronization tool that is used to simplify the solution to the critical section problem (To avoid race condition). A Semaphore s is an integer variable that can only be accessed via two indivisible (atomic) operations namely wait () and signal ()

All modifications to the integer value of the semaphore in the wait () and signal () operations must be executed separately. When one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Usage: Semaphore is two types

Counting Semaphore

There are the scenarios in which more than one processes need to execute in critical section simultaneously. However, counting semaphore can be used when we need to have more than one process in the critical section at the same time.

The programming code of semaphore implementation is shown below which includes the structure of semaphore and the logic using which the entry and the exit can be performed in the critical section

```
struct Semaphore
{
    int value; // processes that can enter in the critical section simultaneously.
    queue type L; // L contains set of processes which get blocked
}
```

Down (Semaphore S)

```
{
```

```

        SS.value = S.value - 1; //semaphore's value will get decreased when a new
                                //process enter in the critical section
    if (S.value < 0)
    {
        put_process(PCB) in L; //if the value is negative then
        //the process will get into the blocked state.
        Sleep();
    }
    else
        return;
}

up (Semaphore s)
{
    SS.value = S.value+1; //semaphore value will get increased when
    //it makes an exit from the critical section.
    if(S.value <= 0)
    {
        select a process from L; //if the value of semaphore is positive
        //then wake one of the processes in the blocked queue.
        wake-up();
    }
}

```

In this mechanism, the entry and exit in the critical section are performed on the basis of the value of counting semaphore. The value of counting semaphore at any point of time indicates the maximum number of processes that can enter in the critical section at the same time.

A process which wants to enter in the critical section first decrease the semaphore value by 1 and then check whether it gets negative or not. If it gets negative then the process is pushed in the list of blocked processes (i.e. q) otherwise it gets enter in the critical section.

Binary Semaphore or Mutex

In counting semaphore, Mutual exclusion was not provided because we have the set of processes which required to execute in the critical section simultaneously. However, Binary Semaphore strictly provides mutual exclusion. Here, instead of having more than 1 slots available in the critical section, we can only have at most 1 process in the critical section. The semaphore can have only two values, 0 or 1.

Let's see the programming implementation of Binary Semaphore.

Struct Bsemaphore

```
{
enum Value(0,1); //value is enumerated data type which can only have two
values 0 or 1.
```

```
Queue type L;
```

```
}
```

```
/* L contains all PCBs corresponding to process
```

```
Blocked while processing down operation unsuccessfully.
```

```
*/
```

Down (Bsemaphore S)

```
{
```

```
if (s.value == 1) // if a slot is available in the
```

```
    //critical section then let the process enter in the queue.
```

```
{
```

```
S.value = 0; // initialize the value to 0 so that no other process can read it as 1.
```

```
}
```

```
else
```

```
{
```

```
    put the process (PCB) in S.L; //if no slot is available
```

```
    //then let the process wait in the blocked queue.
```

```
sleep();
```

```
}
```

```
}
```

Up (Bsemaphore S)

```
{
```

```
if (S.L is empty) //an empty blocked processes list implies that no process
```

```
    //has ever tried to get enter in the critical section.
```

```
{
```

```
S.Value =1;
```

```
}
```

```
else
```

```
{
```

```
Select a process from S.L;
```

```
Wakeup(); // if it is not empty then wake the first process of the blocked queue.
```

```
}
```

```
}
```

Classic Problems of Synchronization:

- Bounded-Buffer Problem or producer consumer problem
- Readers and Writers Problem
- Dining-Philosophers Problem

To avoid the occurrence of race condition, we present a solution to the bounded buffer problem using semaphores. The biggest advantage of this solution using semaphores is that it not only avoids the occurrence of race condition but also allows having size items in the buffer at the same time, thus, eliminating the shortcomings of the solutions using shared memory.

We assume that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The empty and full semaphores count the number of empty and full buffers. The semaphore empty is initialized to the value n ; the semaphore full is initialized to the value 0.

The code for the producer process is:

```
Define n=100;
Int buffer[n];
Csemaphore empty=n;
Full=0;
Bsemaphore mutex =1;
Void produce (void)
{
  Int itemp,in=0;
  Produce_item(itemp)
  While(true)
  {
    Down(empty);
    Down(mutex);
    // add next p to buffer
    Buffer[in]=itemp;
    In=(in+1)mod n;
    up (mutex);
    up (full);
  }
}
```

The code for the consumer process is:

```
Void consumer(void)
{
  Int itemc,out=0;
  While(true)
  {
    down (full);
    down (mutex);
    // remove an item from buffer to next c
    Itemc=buffer[out];
    Out=(out+1)mod n;
    up (mutex);
    up (empty);
    // consume the item in nextc
    Process_item(itemc)
  }
}
```

We can interpret this code as the producer producing full buffers for the consumer or as the consumer producing empty buffers for the producer.

The Readers-Writers Problem:

Concurrently executing processes that are sharing a data object, such as a file or a variable, fall into two groups: readers and writers. The processes in the readers group want only to read the contents of the shared object & the processes in writers group want to update (read and write) the value of shared object. There is no problem if multiple readers access the shared object simultaneously. However, if a writer and some other process (either a reader or writer) access the shared object simultaneously, data may become inconsistent.

To guarantee that such a problem does not occur, when a Writer is accessing the shared object, no reader or writer accesses that shared object. This synchronization problem is termed as readers-writers problem has several variations.

First reader-writer problem requires that no reader will be kept waiting excepting a writer gives permission to use the shared object. No reader should wait for others readers to finish simply because a writer is waiting.

Second readers-writers problem if a writer is ready, that writer performs its write as soon as possible. if a writer is waiting to access the object, no new readers may start reading.

A solution to problem can result in starvation. In the first case writers may starve, in the second case readers may starve. A solution to the first readers-writers problem. The reader process share the following data structures:

semaphore mutex, DB or wrt;

int read_count;

The semaphore mutex and DB are initialized to 1 and read_count is initialized to 0.

The semaphore DB is common to both reader and writer processes. The mutex semaphore is used to ensure mutual exclusion when the variable read_count is updated. The read_count variable keeps track of how many processes are currently reading the object. The semaphore DB functions as a mutual-exclusion semaphore for the writers. It is also used by the first or last reader that enters or exist the critical section. It is not used by readers who enter or exit while other readers are in their critical section.

If a writer is in the critical section and n readers are waiting, then one reader is queued on DB and n-1 readers are queued on mutex. Also, observe that, when a writer executes signal(DB), we may resume the execution of either the waiting readers or a single waiting writer.

The structure of a writer process.

Semaphore mutex=1;

Db=1;

Read count=0;

Void writer(void)

```

While(true)
{
Down (DB);
// writing is performed
up (DB);
}

```

The structure of a Reader process.

```

Void Reader(void)
While(true)
{
Down (mutex);
read_count+ + ;
if (read_count ==1)
Down (DB);
Up (mutex);
// reading is performed
Down (mutex);
read_count --;
if (read_count ==0)
up(wrt);
up (mutex);
}

```

The Dining-Philosophers Problem:

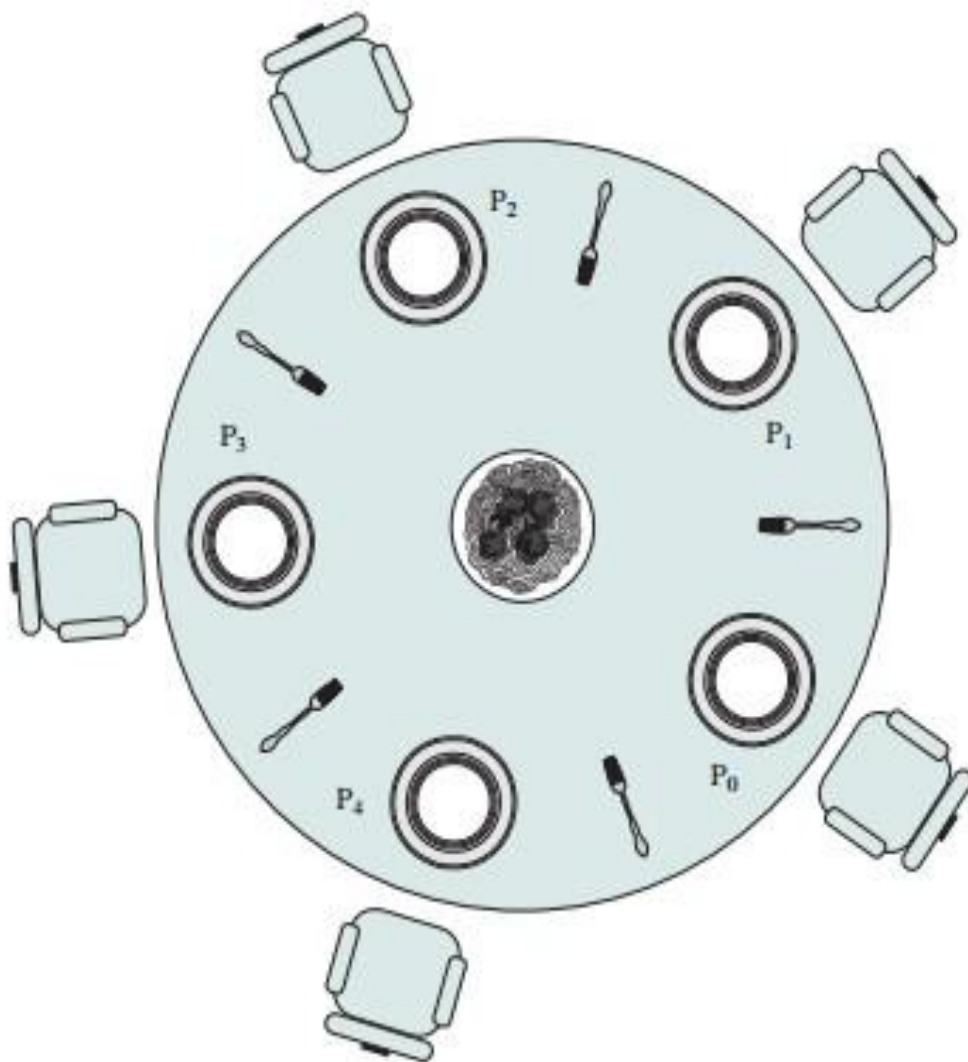
Consider five philosophers sitting around a circular table. There is a bowl of rice in the centre of the table and five chopsticks – one in between each pair of philosophers.

These philosophers spend their lives alternating between two activities: eating and thinking.

When it is time for a philosopher to eat, it must first acquire two chopsticks - one from their left and one from their right.

When a philosopher thinks, it puts down both chopsticks in their original locations

One simple solution is to represent each chopstick with a semaphore. A philosopher tries to take a chopstick by executing a wait () operation on that semaphore; she releases her chopsticks by executing the signal () operation on the correct semaphores. Thus, the shared data are Semaphore chopstick [5]; where all elements of chopstick are initialized to 1. This solution is rejected as it could create a deadlock. Suppose that all five philosophers become hungry simultaneously and each take her left chopstick. All the elements of chopstick will now be equal to 0. When each philosopher tries to take her right chopstick, she will be delayed forever.



```
while (TRUE)
{
wait (chopstick [i] );
wait (chopstick [(i+1) % 5 ] );
// eat
signal (chopstick [i] );
signal (chopstick [(i+1) % 5 ] );
// think
}
```

Some potential solutions to the problem include:

Only allow four philosophers to eat dinner at the same time. (Limited simultaneous processes.)

Allow philosophers to pick up chopsticks only when both are available, in a critical section. (All or nothing allocation of critical resources.)

Use an asymmetric solution, in which odd philosophers pick up their left chopstick first and even philosophers pick up their right chopstick first.

A deadlock-free solution to the dining philosopher's problem does not automatically guarantee a starvation-free one.